Eleição de Líder pelo Algoritmo do Valentão sobre um Detector de Falhas por *Heartbeat*: Implementação, Verificação e Avaliação Experimental

Theo Maceres Silva Mateus Farha Ribeiro Vinicius Leme Soares
Instituto de Computação, Universidade Estadual de Campinas (UNICAMP)
MC714 — Sistemas Distribuídos — 2º semestre de 2026

***Resumo*—Este trabalho implementa dois algoritmos distribuídos complementares, um detector de falhas por *heartbeat* e a eleição de líder pelo algoritmo do valentão (Bully), executados como processos independentes que se comunicam apenas por HTTP/JSON e orquestrados por Docker Compose v2. Para cada algoritmo declaramos o modelo temporal, as propriedades dos canais, os tipos de falha e as propriedades de segurança e vivacidade, que são conferidas por testes automatizados e por um verificador reproduzível sobre os logs. Variando o número de nós em quatro níveis e a perda de mensagens em quatro níveis, com dez repetições cada, a detecção ficou entre 1,5 e 2,5 s como previsto ($p_{95} \approx 2{,}47$ s), a convergência do Bully após a queda do líder ficou abaixo de 2,5 s sem perda e a carga de *heartbeats* seguiu exatamente $n(n-1)/\Delta t$. Com 30% de perda, a convergência dentro da janela caiu para 80% e surgiram falsos positivos e violações de acordo. Como extras, avaliamos cinco cenários de recuperação, partição e falhas combinadas, nos quais a partição produz *split-brain* de 3 a 8 s, e integramos os dois algoritmos num serviço de tarefas tolerante a falhas que confirmou 100% das 6000 tarefas, com p_{99} de 119 ms sem falhas e de 1572 ms sob quedas.**

***Index Terms*—eleição de líder, algoritmo do valentão, detector de falhas, *heartbeat*, injeção de falhas, *split-brain*.**

I. Motivação e Escolha dos Algoritmos

Muitos serviços distribuídos dependem de um coordenador: um escalonador de tarefas, um primário de replicação ou um sequenciador. Quando ele cai, os demais precisam perceber a queda e escolher outro, sem intervenção humana. Os dois problemas são separáveis [1]: perceber a queda é tarefa de um detector de falhas [3], e escolher o substituto é tarefa de um algoritmo de eleição [2].

Escolhemos o **detector de falhas por *heartbeat***, do tema de detecção de falhas, e a **eleição pelo algoritmo do valentão (Bully)** [2], do tema de eleição de líder. Os dois se complementam: o Bully supõe que um nó maior que não responde caiu, e o detector fornece justamente essa informação, com propriedades que podem ser medidas. Como itens extras, avaliamos cenários de recuperação, partição e falhas combinadas, e integramos os dois algoritmos num serviço de tarefas com coordenador eleito.

II. Arquitetura e Ambiente

Nós. Cada nó lógico é um processo Python (`python -m src.node`) que usa apenas a biblioteca padrão. Ele expõe um servidor HTTP com três rotas: `/msg`, por onde passam todas as mensagens dos algoritmos; `/control`, pela qual o injetor liga e desliga falhas de rede; e `/status`, que informa o líder corrente. Os algoritmos registram *handlers* por tipo de mensagem (`node.register`) e enviam com `node.net.send(peer, tipo, carga, timeout)`, sempre com *timeout* explícito, já que a chamada bloqueia até a resposta.

Comunicação. Optamos por HTTP/JSON em vez de gRPC ou de um *broker*, por ser simples de depurar e por permitir injetar falhas no próprio emissor. Nenhum componente externo implementa a detecção ou a eleição: o transporte só entrega bytes, e toda a lógica de *heartbeat*, suspeita, `ELECTION`, `OK` e `COORDINATOR` está em `src/detector.py` e `src/bully.py`. Não houve reutilização de código de terceiros; as únicas dependências externas são `pyyaml`, para ler os cenários, e `matplotlib` e `pytest`, para análise e testes.

Observabilidade. Cada nó grava um log estruturado JSONL com relógio lógico de Lamport [5]. Após um reinício, o relógio é retomado do maior valor do log, para não voltar a zero. Todos os eventos usados nas métricas e no verificador (`send`, `recv`, `suspect`, `restore`, `election_start`, `leader`, `election_end`) saem desses logs.

Orquestração. O `compose.yaml` define os nós 1 a 3 (e um perfil com 5 nós), com *healthcheck* e portas publicadas apenas em `127.0.0.1`, pois `/control` não tem autenticação. O script `scripts/env.sh` constrói, sobe, testa (`smoke`) e remove o ambiente, e `scripts/run_scenario.sh` executa um cenário, coleta os logs e roda o verificador.

Injeção de falhas. Adotamos uma abordagem híbrida. Falhas de enlace (perda, atraso, *jitter*, duplicação e partição) são aplicadas no nível da aplicação, em `src/network.py`, o que dispensa privilégios de `tc netem` e funciona igual em contêineres e em testes locais. Cada enlace $i \to j$ tem um gerador pseudoaleatório próprio, semeado com $\texttt{SEED} \cdot 1000 + i$ e o destino, de modo que o tráfego para um nó não altera os sorteios dos outros. Falhas de nó usam SIGKILL (`docker`

`kill`, ou o sinal direto no processo), que mata o nó sem encerramento gracioso, e a recuperação sobe um processo novo, sem o estado em memória. Os cenários são declarados em YAML, com instantes, ações e semente, e executados pelo `FaultInjector`.

III. Modelo do Sistema

Canais. Cada mensagem é uma requisição HTTP independente, com conexão TCP própria e enviada por várias *threads*, portanto *não há FIFO* entre mensagens de um mesmo par. O envio é de melhor esforço, sem retransmissão automática: a perda é sorteada em cada sentido, requisição e resposta, com a mesma probabilidade p, de modo que uma chamada falha com probabilidade $1-(1-p)^2$. A partição descarta a mensagem no emissor, que só percebe a falha no *timeout*, enquanto um nó caído recusa a conexão em milissegundos. Com duplicação, a mesma mensagem chega duas vezes, por isso todos os *handlers* são idempotentes. Supomos que não há corrupção nem mensagens forjadas.

Modelo temporal. Os dois algoritmos supõem um sistema *parcialmente síncrono*. Num sistema assíncrono não é possível distinguir um nó lento de um nó caído, e nenhum *timeout* é seguro [3]. Supomos que, a partir de um instante desconhecido, o atraso fica abaixo de um limite Δ, com $\Delta t + \Delta < T$, em que Δt é o intervalo de *heartbeat* e T o *timeout* de suspeita. As propriedades de segurança do Bully valem nesses períodos síncronos.

Falhas. Consideramos *crash-stop*, *crash-recovery* sem estado e partições simétricas; não consideramos falhas bizantinas.

Relógios e persistência. Os *timeouts* usam só o relógio monotônico local, sem sincronização entre nós; o relógio de Lamport serve apenas para ordenar os logs. Nenhum dos algoritmos persiste estado: o líder é determinado pelos IDs dos nós vivos, e um nó que volta descobre o líder fazendo uma eleição. Um contador de época não ajudaria sem persistência, pois voltaria a zero justamente no nó de maior ID, que é o que volta e vence.

IV. Algoritmos e Propriedades de Correção

A. Detector de falhas por heartbeat

A cada Δt (`HB_INTERVAL`, 0,5 s), cada nó envia um `heartbeat` a cada outro nó, com uma *thread* por destino, de modo que um nó lento não atrasa os demais. O receptor anota o instante de chegada. Uma *thread* de verificação, também a cada Δt, marca como **suspeito** todo nó sem *heartbeat* há mais de T (`HB_TIMEOUT`, 2,0 s), e **restaura** o suspeito que voltou a enviar. Como só essa *thread* muda o estado, os avisos de um mesmo nó saem em ordem. A API oferece `on_suspect`, `on_restore` e `vivos()`.

- **Segurança:** um nó só é suspeito depois de mais de T sem *heartbeat*, e `suspect` e `restore` de um mesmo nó se alternam.
- **Completude (vivacidade):** todo nó que cai é suspeito por todos os corretos em até $T + \Delta t = 2{,}5$ s, pois o último *heartbeat* é anterior à queda e a verificação ocorre a cada Δt.
- **Precisão eventual (vivacidade):** no período síncrono nenhum nó correto é suspeito. Com perda p, um falso positivo exige perder cerca de $k = \lfloor T/\Delta t \rfloor = 4$ *heartbeats* seguidos, com probabilidade da ordem de p^k por janela.
- **Custo:** $n(n-1)$ mensagens por Δt, ou seja, $O(n^2)$.

B. Eleição pelo algoritmo do valentão

Os IDs são únicos e fixos, e o líder é o maior ID correto [2]. O nó i envia `ELECTION` em paralelo a todos os maiores e espera até `RPC_TIMEOUT` (1,0 s). O `OK` é a própria resposta HTTP. Sem nenhum `OK`, i vence e envia `COORDINATOR` a todos; com algum `OK`, espera o `COORDINATOR` por até $2{\cdot}$`RPC_TIMEOUT` e, se ele não chega, recomeça. Quem recebe `ELECTION` responde e inicia a própria eleição; quem recebe `COORDINATOR` de um nó menor o rejeita e também inicia uma eleição. Uma eleição começa quando o nó não tem líder, quando o detector suspeita do líder ou quando um nó maior que o líder está vivo para o detector. Essas condições são conferidas a cada Δt e também nos *callbacks* do detector, o que substitui a retransmissão: enquanto a situação estiver errada, a eleição é refeita.

- **Segurança (acordo):** no período síncrono, se nenhum nó cai ou volta durante a eleição, todo nó que decide um líder decide o maior ID correto. O vencedor não recebeu `OK` de nenhum maior, e como todo correto responde a tempo, não há maior correto.
- **Vivacidade (convergência):** cessadas as quedas e com a rede síncrona, todos os corretos acabam com o maior ID correto como líder, em até $T + \Delta t + 2{\cdot}$`RPC_TIMEOUT` $= 4{,}5$ s após a queda do líder.
- **Complexidade:** no melhor caso, só o nó $n-1$ inicia, com 1 `ELECTION` e $n-1$ `COORDINATOR`, $O(n)$. No pior caso de uma rodada, o nó 1 inicia, somando $n(n-1)/2$ `ELECTION`, $(n-1)(n-2)/2$ `OK` e $n-1$ `COORDINATOR`, ou $n(n-1)$ mensagens, $O(n^2)$.

Uma particularidade da composição com o detector é que os detectores não suspeitam do líder caído todos ao mesmo tempo. O novo líder, que vence logo, ainda vê o antigo vivo por até um Δt e repete a eleição (motivo `no_maior_vivo`). Essas rodadas extras não mudam o líder nem o limite de tempo, mas fazem o custo medido passar da fórmula de uma rodada (Seção VII).

V. Testes e Verificação dos Logs

A suíte `pytest` tem três camadas, executadas em cerca de 1 minuto e em integração contínua a cada *push*. Os testes **unitários** chamam *handlers* diretamente, com estado escolhido à mão (por exemplo, rejeitar `COORDINATOR` de nó menor). Os de **integração** sobem nós reais em portas livres de `127.0.0.1`. Os de **falha** injetam queda, perda, atraso, duplicação e partição. Cada algoritmo tem um cenário normal e um de falha: para o detector, `test_rede_estavel_sem_falso_positivo` (3 nós, 20 intervalos, nenhuma suspeita) e `test_crash_detectado_por_todos_e_recuperacao`; para o Bully, `test_`

`cenario_normal_lider_e_o_maior_id` (4 nós com eleições concorrentes) e `test_lider_cai_novo_lider_e_anunciado_e_maior_id_volta`, que também confere que os contadores de mensagens batem com os eventos do log.

O **verificador** (`analysis/log_verifier.py`) lê os logs de qualquer execução, inclusive no *compose*, divide cada arquivo em vidas do nó (de `node_start` ao fim) e emite `[OK]` ou `[AVISO]` para seis propriedades: `detector.suspeita_apos_timeout`, `detector.alternancia`, `detector.completude`, `detector.restauracao`, `bully.acordo` e `bully.convergencia`. Os parâmetros saem do próprio log, e os prazos de vivacidade admitem folga de 0,5 s para o escalonamento das *threads*. Para validar o próprio verificador, um teste parte de um log sintético correto e quebra uma propriedade por vez, exigindo que ele acuse exatamente as quebradas; na direção oposta, execuções corretas, inclusive com SIGKILL e linha final cortada, não podem gerar aviso. Um `[OK]` mostra que aquela execução respeitou as propriedades, e não que todas respeitam.

VI. Metodologia Experimental

Ambiente. Os experimentos rodam com um processo por nó (o mesmo comando do contêiner) em `127.0.0.1`, sem o *compose*, o que torna a subida e a derrubada mais rápidas e permite reproduzir tudo com `python -m experiments.run_all`. A máquina foi um Apple M2 Pro (10 núcleos, 16 GB), macOS 26.7.1 e CPython 3.14.4. Cada CSV vem com um `.meta.json` que registra comando, *commit* (sem alterações locais), início, fim, parâmetros, ambiente e a descrição das colunas.

Roteiro de um ensaio. Sobem n nós; espera-se todos terem o nó n como líder e mais 2 s de **aquecimento**, fora das medidas; mede-se 4 s sem falha; o líder recebe SIGKILL e mede-se mais 10 s. Cada ensaio dura cerca de 17 s. Fixamos $\Delta t = 0{,}5$ s, $T = 2{,}0$ s e `RPC_TIMEOUT`$= 1{,}0$ s, os padrões do *compose*.

Grades. Variamos o número de nós, $n \in \{3{,}5{,}7{,}9\}$ sem perda, e a perda, $p \in \{0;\ 0{,}1;\ 0{,}2;\ 0{,}3\}$ com $n = 5$. Cada nível foi repetido 10 vezes, com a repetição r usando `SEED`$= r$; as repetições foram intercaladas entre níveis, para que uma variação de carga da máquina não caia num nível só. As grades duraram 671,5 s e 694,7 s. Sem perda, a semente não altera nada, e a variabilidade vem do escalonamento e da fase da queda em relação aos *heartbeats*. Não variamos atraso nem duplicação: o atraso injetado ocorre antes da chamada e não conta no *timeout*, e a duplicação não é combinada com perda.

Métricas. Para o detector: o *tempo de detecção*, da queda ao `suspect` de cada sobrevivente, que é a latência do serviço de detecção; a *taxa de falsos positivos* por par monitorado por minuto, a métrica de qualidade de Chen, Toueg e Aguilera [4]; e os *heartbeats/s*, que medem o custo $O(n^2)$. Para o Bully: o *tempo de convergência*, da queda até todos os sobreviventes terem o nó $n-1$ como líder, que é o tempo de indisponibilidade do coordenador; e o *número de mensagens* `ELECTION`, `OK` e `COORDINATOR`, somado dos `election_end` de todos os nós. Reportamos média com intervalo de confiança de 95% pela distribuição t de Student e, para as latências, os percentis p_{50}, p_{95} e p_{99}.

Tabela I
Grade de nós, sem perda: média ± semi-IC de 95% de dez repetições. Mensagens do Bully medidas contra o pior caso de uma rodada.

n	Detecção (s)	Converg. (s)	Msgs Bully	$n(n-1)$	HB/s	$\frac{n(n-1)}{\Delta t}$
3	$1{,}90 \pm 0{,}21$	$1{,}86 \pm 0{,}16$	$5{,}7 \pm 1{,}2$	6	11,6	12
5	$2{,}32 \pm 0{,}16$	$1{,}98 \pm 0{,}01$	$26{,}4 \pm 7{,}4$	20	40,0	40
7	$2{,}23 \pm 0{,}18$	$1{,}95 \pm 0{,}01$	$67{,}0 \pm 28{,}2$	42	83,9	84
9	$2{,}25 \pm 0{,}14$	$1{,}94 \pm 0{,}13$	$120{,}7 \pm 46{,}1$	72	143,9	144

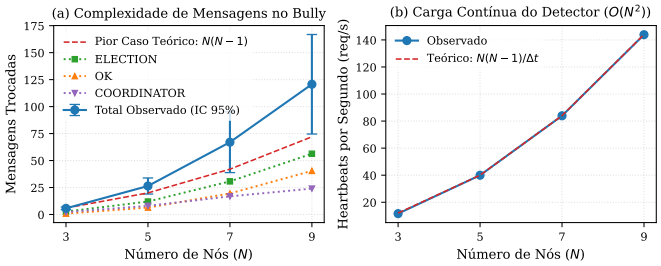


Figura 1. (a) Mensagens do Bully após a queda do líder, por tipo, com IC de 95% no total, contra o pior caso de uma rodada; (b) *heartbeats* por segundo medidos contra $n(n-1)/\Delta t$.

VII. Resultados

A. Escala

A Tabela I resume a grade de nós. O **tempo de detecção** não depende de n, como esperado para um detector em que cada par é monitorado de forma independente: juntando as amostras de cada sobrevivente, o p_{50} fica entre 1,88 e 2,00 s e o p_{95} entre 2,04 e 2,47 s, sempre dentro da faixa prevista $[T - \Delta t, T + \Delta t] = [1{,}5;\ 2{,}5]$ s. A variação dentro dessa faixa é a fase da queda em relação ao último *heartbeat* e à verificação seguinte. As amostras de um mesmo ensaio não são independentes, pois a queda é a mesma; por isso o p_{99} coincide praticamente com o máximo.

A **convergência** ficou entre 1,86 e 1,98 s em média, com máximo de 2,46 s em $n = 9$, bem abaixo do limite de 4,5 s. Ela é menor que a detecção média porque não precisa esperar todos os detectores: basta que o nó $n-1$ suspeite, vença e anuncie. A eleição em si, da primeira suspeita à convergência, levou de 7 a 36 ms em mediana, porque o envio a um processo morto é recusado na hora. Nenhum ensaio da grade gerou `[AVISO]` no verificador.

A Fig. 1(b) mostra que a carga do detector segue exatamente $n(n-1)/\Delta t$, de 12 a 144 mensagens/s: é o custo contínuo de $O(n^2)$, pago mesmo sem falhas. Já as mensagens do Bully, na Fig. 1(a), crescem de forma superquadrática e passam do pior caso de uma rodada a partir de $n = 5$, com grande variabilidade. Isso decorre das rodadas extras descritas na Seção IV: o número médio de eleições iniciadas após a queda subiu de 1,9 em $n = 3$ para 15,2 em $n = 9$, pois com mais nós há mais detectores que, ainda vendo o líder antigo vivo,

Tabela II
GRADE DE PERDA, $n = 5$: MÉDIA $\pm$ SEMI-IC DE 95% E PERCENTIS DA CONVERGÊNCIA.

p	Detecção (s)	Converg. (s)	p_{50}/p_{95} conv.	Conv.	FP/par·min
0,0	$2{,}12 \pm 0{,}17$	$1{,}98 \pm 0{,}01$	1,98 / 1,99	100%	0,00
0,1	$2{,}13 \pm 0{,}16$	$2{,}18 \pm 0{,}46$	1,98 / 3,11	100%	0,00
0,2	$2{,}04 \pm 0{,}19$	$3{,}01 \pm 1{,}17$	3,06 / 5,03	100%	0,57
0,3	$2{,}10 \pm 0{,}30$	$3{,}43 \pm 2{,}09$	3,25 / 6,96	80%	0,78

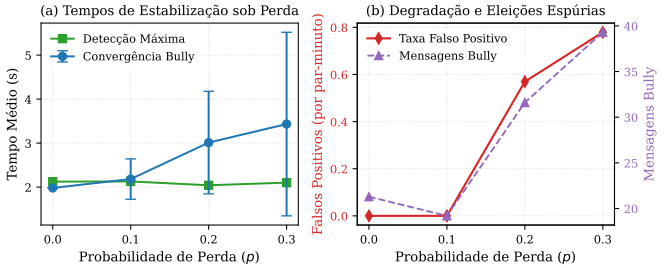


Figura 2. (a) Detecção máxima e convergência sob perda, com IC de 95%; (b) taxa de falsos positivos e mensagens do Bully.

refazem a eleição. Cada uma custa $O(n)$ mensagens, e como seu número também cresce com n, o total cresce mais rápido que $n(n-1)$. O líder eleito e o tempo de convergência não mudam, mas o custo real do Bully acoplado a um detector com suspeitas assíncronas fica acima do previsto pela análise de uma rodada isolada.

B. Perda de mensagens

A Tabela II e a Fig. 2 mostram que a perda afeta os dois algoritmos de forma muito diferente. A **detecção da queda real** praticamente não muda, em torno de 2,1 s, porque depende de um silêncio total que a perda não altera; ela pode até ser antecipada por *heartbeats* perdidos logo antes da queda.

Já a **precisão** do detector se degrada: sem falsos positivos até $p = 0{,}1$, a taxa salta para 0,57 e 0,78 por par por minuto em $p = 0{,}2$ e $0{,}3$. Isso é coerente com a dependência p^k: com $p = 0{,}1$, perder 4 *heartbeats* seguidos tem probabilidade 10^{-4} por janela, enquanto com $p = 0{,}3$ ela já é $8 \cdot 10^{-3}$, e com 120 janelas por minuto a estimativa fica na mesma ordem de grandeza da taxa medida.

O **Bully** é o mais afetado. Ele não retransmite: um `ELECTION`, um `OK` ou um `COORDINATOR` perdido só é corrigido na próxima conferência periódica, e cada chamada falha com probabilidade $1-(1-p)^2$, ou 51% em $p = 0{,}3$. A convergência média sobe de 1,98 para 3,43 s, o p_{95} passa de 1,99 para 6,96 s e, em $p = 0{,}3$, dois dos dez ensaios não convergiram nos 10 s de observação. Além disso, os falsos positivos disparam eleições espúrias, o que aumenta as mensagens e explica por que, em parte dos ensaios, a convergência veio antes de algum sobrevivente suspeitar da queda real. Em $p \geq 0{,}2$ o verificador acusou `bully.acordo` em 7 e 9 ensaios, respectivamente, e `bully.convergencia` em 3 e 4. São violações esperadas: o acordo só é garantido no período síncrono, e sob perda a convergência ocorre com probabilidade 1, mas sem limite de tempo. As quatro propriedades do detector deram `[OK]` em todos os ensaios.

Tabela III
CENÁRIOS DE FALHA, 3 NÓS, 5 REPETIÇÕES: TEMPO COM MAIS DE UM LÍDER (MÉDIA $\pm$ SEMI-IC DE 95%), RECONVERGÊNCIA APÓS A CURA OU VOLTA (MEDIANA) E PROPRIEDADES COM AVISO.

Cenário	2 líderes (s)	Reconv. (s)	Aviso
Queda e recuperação	$0{,}00 \pm 0{,}00$	0,20	nenhum
Cascata e recuperação	$0{,}02 \pm 0{,}01$	0,19 a 0,21	nenhum
Partição do líder	$2{,}73 \pm 0{,}55$	0,99	`bully.acordo`
Partição do menor ID	$3{,}84 \pm 0{,}33$	0,99	`bully.acordo`
Partição, queda e perda	$7{,}85 \pm 0{,}33$	0,55	`bully.acordo`

C. Recuperação, partição e falhas combinadas (extra)

Avaliamos cinco cenários com 3 nós e 5 repetições cada (535,2 s no total). Em `queda_e_recuperacao` cai e volta um nó que não é o líder, com atraso de 100 ms no meio; em `cascata_e_recuperacao` o líder e o próximo líder caem com 1 s de diferença e depois voltam; em `particao_rede` o líder é isolado por 5 s; em `particao_menor_id` o nó de menor ID é isolado por 6 s; e em `particao_queda_e_perda` o líder é isolado, todos passam a perder 20% das mensagens e o líder do outro lado cai durante a partição.

A Tabela III resume os resultados. A **recuperação** de um nó, do comando de reinício até todos terem o maior vivo como líder, levou de 0,16 a 0,38 s, quase tudo para subir o processo Python. Quando o nó que volta é o de maior ID, ele retoma a liderança e há dois líderes por cerca de 20 ms, até o `COORDINATOR` chegar. Na **cascata**, a segunda queda chega antes de a primeira ser percebida; o nó 1 suspeita do 3, o 2 recusa a conexão na hora e o 1 vence sem esperar *timeout*, de modo que a segunda queda não custou tempo extra.

Toda **partição** produziu *split-brain* em 5 de 5 repetições, e `bully.acordo` foi violado. O lado sem o maior ID não recebe resposta dele e o trata como caído; o Bully não tem quórum, e até um nó sozinho se elege. O tempo com dois líderes é aproximadamente a duração da partição, menos a detecção e o *timeout* do `ELECTION` descartado (cerca de 3 s), mais a reconvergência após a cura, de 0,5 a 1,0 s, que é o tempo de o detector receber o primeiro *heartbeat* e verificar. No cenário combinado há dois períodos de *split-brain*, primeiro com os líderes 2 e 3 e depois com 1 e 3, somando quase 8 s. As quatro propriedades do detector e a convergência após a cura se mantiveram em todos os cenários: para o detector, partição e queda são o mesmo silêncio.

D. Integração: serviço de tarefas (extra)

Os dois algoritmos foram integrados num escalonador de tarefas, sem algoritmo novo. O líder do Bully é o coordenador e distribui as tarefas, em rodízio, aos *workers* que o detector vê vivos (`vivos()`); um `EXEC` sem resposta ou recusado é reatribuído a outro *worker*. O cliente repete cada tarefa até a confirmação e procura o novo líder quando o atual cai. O *worker* só executa para o próprio líder, o que impede que um líder antigo use um *worker* que já soube da troca. Dois

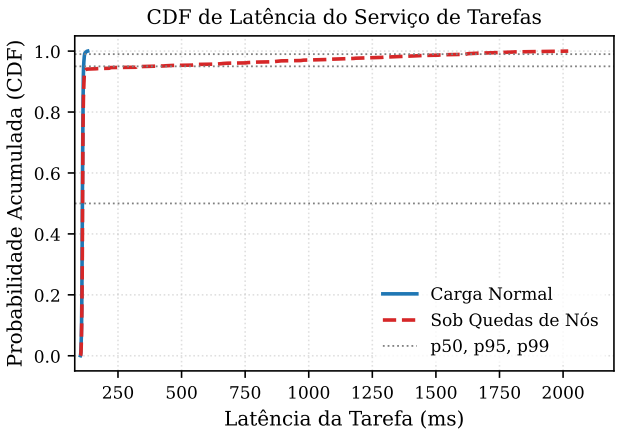


Figura 3. CDF da latência das 3000 tarefas de cada cenário do serviço integrado.

requisitos são conferidos pelo verificador: `tarefas.execucao` (vivacidade: toda tarefa é confirmada, e só depois de executada) e `tarefas.um_lider` (segurança: nenhuma tarefa fica com dois líderes ao mesmo tempo). A semântica é "pelo menos uma vez" [1]: sem persistência, uma tarefa em execução quando o líder cai é reenviada pelo cliente.

Com 3 nós, carga aberta de 20 tarefas/s de 100 ms por 30 s e 5 repetições por cenário, comparamos uma execução sem falhas com outra em que o *worker* 1 cai aos 5 s e volta aos 10 s, e o líder 3 cai aos 15 s e volta aos 22 s.

As 6000 tarefas foram confirmadas, a vazão igualou a carga oferecida (19,96 tarefas/s) e o verificador não acusou nenhuma propriedade. Sem falhas, a latência é a tarefa mais cerca de 10 ms de duas idas e voltas HTTP, com $p_{50}/p_{95}/p_{99}$ de 110,1/114,8/118,0 ms. Sob quedas, o p_{50} quase não muda (111,0 ms), mas a cauda cresce para 372,7 ms no p_{95} e 1571,8 ms no p_{99} (Fig. 3). A **queda do *worker*** não altera a latência: até a suspeita, cerca de 20 tarefas são enviadas a ele, recusadas na hora e reatribuídas. Toda a cauda vem da **queda do líder**, que deixa o serviço cerca de 1,76 s sem confirmações, o tempo de detecção mais uma eleição de poucos milissegundos. Nas 5 execuções, as 2 tarefas em execução nesse instante rodaram duas vezes, o custo esperado de "pelo menos uma vez". A volta do líder não teve custo visível para o cliente.

VIII. Limitações

Domínios de falha. Todos os nós rodam na mesma máquina e compartilham CPU, memória, *kernel* e relógio. Uma falha do hospedeiro derruba todos, e com $n = 9$ o escalonamento de 9 processos e 144 *heartbeats*/s entra nas medidas. O relógio compartilhado também é o que permite ao verificador comparar instantes de logs de nós diferentes; em máquinas separadas seria preciso sincronizá-los.

Processos em vez de contêineres. Nos experimentos, um envio ao nó morto é recusado na hora. No *compose*, dependendo da rede do Docker, o `ELECTION` ao líder caído pode esperar até o `RPC_TIMEOUT`, o que aumentaria a convergência, mas não a detecção.

Algoritmos. O Bully viola o acordo sob partição; evitá-lo exigiria quórum de maioria ou épocas persistidas, como no Raft [6], o que está fora do escopo: o objetivo foi medir a consequência. Os *timeouts* são fixos, sem ajuste adaptativo, e a detecção espera T mesmo quando a conexão é recusada. Não há persistência, e por isso o serviço integrado oferece "pelo menos uma vez", e não "exatamente uma vez".

Experimentos. Atraso e duplicação não foram variados, só partições simétricas foram testadas e o serviço integrado não foi avaliado sob partição. A convergência tem uma amostra por ensaio, 10 por nível, o que torna o p_{99} praticamente igual ao máximo, e os cenários têm 5 repetições. Na grade de perda, a semente reproduz as taxas de perda, mas não exatamente quais mensagens são perdidas, pois várias *threads* compartilham cada enlace.

IX. Conclusão

O detector de falhas e o Bully se comportaram como o modelo prevê no período síncrono. A detecção ficou dentro de $[T - \Delta t, T + \Delta t]$ independentemente de n, a convergência do Bully ficou abaixo de 2,5 s contra um limite de 4,5 s, e as seis propriedades verificadas nos logs valeram em todos os ensaios sem perda. A carga do detector seguiu exatamente $n(n-1)/\Delta t$, enquanto o custo do Bully superou o pior caso de uma rodada por causa das eleições repetidas pelos nós que ainda viam o líder antigo vivo, um efeito que só aparece quando os dois algoritmos são compostos.

Fora do período síncrono, as garantias se degradam conforme o modelo: a perda quase não afeta a detecção da queda real, mas gera falsos positivos a partir de $p = 0{,}2$, e o Bully, que não retransmite, passa a convergir mais devagar, com aviso de acordo na maioria dos ensaios e, em $p = 0{,}3$, sem convergir em 20% deles. A partição é o caso extremo: o Bully, sem quórum, sempre produz *split-brain*, que só termina após a cura. Por fim, integrados num serviço de tarefas, os dois algoritmos bastaram para mascarar as quedas de *worker* e de líder, ao preço de uma interrupção de cerca de 1,8 s e de duas execuções repetidas por queda do líder.

Referências

[1] M. van Steen e A. S. Tanenbaum, *Distributed Systems*, 4th ed. distributed-systems.net, 2023.
[2] H. Garcia-Molina, "Elections in a distributed computing system," *IEEE Trans. Comput.*, vol. C-31, no. 1, pp. 48-59, 1982.
[3] T. D. Chandra e S. Toueg, "Unreliable failure detectors for reliable distributed systems," *J. ACM*, vol. 43, no. 2, pp. 225-267, 1996.
[4] W. Chen, S. Toueg e M. K. Aguilera, "On the quality of service of failure detectors," *IEEE Trans. Comput.*, vol. 51, no. 5, pp. 561-580, 2002.
[5] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558-565, 1978.
[6] D. Ongaro e J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX ATC*, 2014, pp. 305-319.